

Ejercicio POO

Se desea desarrollar un sistema para un juego multijugador masivo en línea (MMO). En este sistema, los jugadores pueden controlar diferentes personajes dentro de un servidor del juego. Se pide implementar una solución en Java utilizando programación orientada a objetos con herencia y colecciones.

Requisitos

1. Clase **Personaje**

Atributos:

- **nombre** (String)
- **nivel** (int)
- **vida** (int)
- **velocidad** (int)

Métodos:

- Constructor para inicializar los atributos.
- Método **atacar(Personaje objetivo)**: Disminuye los puntos de vida del objetivo (implementar lógicamente más adelante).
- Método **recibirGolpe()**: Reduce sus puntos de vida y devuelve true si estos bajan de 0.
- Método **toString()**: Imprime los datos del personaje.

2. Clase **Guerrero** o **Arquero** (hereda de **Personaje**)

Elige una de las dos clases y crea su implementación.

Atributos adicionales:

- Para **Guerrero**: **fuerza** (int)
- Para **Arquero**: **precision** (int)

Métodos:

- Sobreescibir **atacar(Personaje objetivo)**, aplicando daño según el atributo específico. Para el guerrero el daño se calculará a partir del producto de la fuerza por su nivel dividido entre sus puntos de vida. Para el arquero el daño se calculará como un porcentaje del nivel usando la precisión. A los arqueros se les hace el doble de daño.
- Sobreescibir **toString()**.

3. Clase **Servidor**

Atributos:

- **nombre** (String)
- **personajes** (Colección de personajes)

Métodos:

- `agregarPersonaje(Personaje p)`: Añade un personaje a la colección.
- `listarPersonajes()`: Muestra los personajes registrados en el servidor.
- `ordenarPersonajes()`: Ordena los personajes dentro de la estructura de datos de manera ascendente utilizando el atributo de velocidad. Implementa para ello cualquier algoritmo de ordenación.
- `batalla(Personaje p1, Personaje p2)`: Simula una pelea entre dos personajes, reduciendo sus puntos de vida. El método acaba cuando la vida de alguno de ellos sea de cero o menos.